Chapter 25:
Programming paradigms

Learning objectives

By the end of this chapter you should be able to:

- show understanding of what is meant by a programming paradigm
- show understanding of the characteristics of a number of programming paradigms:
  - low-level, imperative, object-oriented and declarative.

25.01 Programming paradigms

A **programming paradigm** is a fundamental style of programming. Each paradigm will support a different way of thinking and problem solving. Paradigms are supported by programming language features. Some programming languages support more than one paradigm. There are many different paradigms, and some overlap. Here are just a few different paradigms.



KEY TERM

Programming paradigm: A fundamental style of programming

Low-level programming paradigm

The features of low-level programming languages give us the ability to manipulate the contents of memory addresses and registers directly and exploit the architecture of a given processor. We solve problems in a very different way when we use the low-level programming paradigm than if we use a high-level paradigm. See Chapter 6 and Chapter 28 for low-level programming examples. Note that each different type of processor has its own programming language. There are 'families' of processors that are designed with similar architectures and therefore use similar programming languages. For example, the Intel processor family (present in many PC-type computers) uses the x86 instruction set.

Imperative programming paradigm

Imperative programming involves writing a program as a sequence of explicit steps that are executed by the processor. Most of the programs in this book use imperative programming (Chapters 12 to 15 and Chapters 23, 24 and 26). An imperative program tells the computer *how* to get a desired result, in contrast to declarative programming where a program describes *what* the desired result should be. Note that the procedural programming paradigm belongs to the imperative programming paradigm. There are many imperative programming languages, Pascal, C and Basic to name just a few.

Object-oriented programming paradigm

The object-oriented paradigm is based on objects interacting with one another. These objects are data structures with associated methods (see Chapter 27). Many programming languages that were originally imperative have been developed further to support the object-oriented paradigm. Examples include Pascal (under the name Delphi or Object Pascal) and Visual Basic (the .NET version being the first fully object-oriented version). Newer languages, such as Python and Java, were designed to be object-oriented from the beginning.

Declarative programming paradigm

Programming languages such as Pascal, VB and Python are referred to as 'imperative programming languages' because the programmer writes sequences of statements that reflect how to solve the problem. When a programmer uses a declarative programming language, the programmer writes down (in the language of logic) a declarative specification that describes the situation of interest: *what* the problem is. The programmer doesn't tell the computer what to do. To get information, the programmer poses a query (sets a goal). It's up to the logic programming system to work out how to get the answer.

Declarative programs are expressed as formal logic and computations are deductions from the formal logic statements (see Chapter 29). Declarative programming languages include SQL (see Chapter 11) and Prolog (Chapter 29).

Reflection Point:

Why are there different paradigms?

After you have studied Chapters 26 to 29 do Tasks 25.01 and 25.02

TASK 25.01

Draw a line between a statement on the left and its matching programming paradigm on the right.

Commands available depend on the type of processor	Declarative
Information hiding is used to protect internal properties of an object	
Statements operate on data	Imperative
Answer a question via a search for a solution	
Data is combined with procedures	Low-level
First do this and then do that	
The basic concept is a relation	Object-oriented

TASK 25.02

For each of the four programming paradigms in this chapter, give one programming statement example that is characteristic for this paradigm. State the programming language you used for each example.

Chapter 27:
Object-Oriented Programming (OOP)

Learning objectives

By the end of this chapter you should be able to:

- show understanding of the terminology associated with OOP: objects, properties, methods, classes, inheritance, polymorphism, containment/aggregation, encapsulation, getters, setters and instances
- show understanding of how to solve a problem by designing appropriate classes
- show understanding of and write code that demonstrates the use of OOP.

27.01 Concept of OOP

Chapters 14 and 26 covered programming using the procedural aspect of our programming languages. Procedural programming groups related programming statements into subroutines. Related data items are grouped together into record data structures. To use a record variable, we first define a record type. Then we declare variables of that record type.

Example of using a record

A car manufacturer and seller wants to store details about cars. These details can be stored in a record structure (see Chapter 16, Section 16.01 and Chapter 26, Section 26.02):

```
TYPE CarRecord
    DECLARE VehicleID           : STRING
    DECLARE Registration        : STRING
    DECLARE DateOfRegistration  : DATE
    DECLARE EngineSize          : INTEGER
    DECLARE PurchasePrice       : CURRENCY
ENDTYPE
```

We can write program code to access and assign values to the fields of this record. For example:

```
PROCEDURE UpdateRegistration(BYREF ThisCar : CarRecord, BYVALUE NewRegistration)
    ThisCar.Registration ← NewRegistration
ENDPROCEDURE
```

We can call this procedure from anywhere in our program. This seems a well-regulated way of operating on the data record. However, we can also access the record fields directly from anywhere within the scope of `ThisCar`:


```
ThisCar.EngineSize ← 2500
```

Classes in OOP

OOP goes one step further and groups together the data structure and the subroutines that operate on the data items in this data structure. Such a group is called an **object**. The data of an object are called **attributes** and the subroutines acting on the attributes are called **methods**. The idea behind OOP is that attributes can only be accessed through methods. The direct path to the data is unavailable. Attributes are referred to as 'private'. The methods to access the data are made available to programmers, so these are 'public'. The feature of data being combined with the subroutines acting on this data is known as **encapsulation**. To use an object, we first define an object type. An object type is called a **class**.

Classes are templates for objects. When a class type has been defined it can be used to create one or more objects of this class type. Therefore, an object is an instance of a class.

KEY TERMS

Object: an instance of a class

Attributes: the data items of a class (also sometimes referred to as fields)

Methods: the subroutines of a class

Encapsulation: combining data and subroutines into a class and restricting external access to the data

Class: a type that combines a data structure with the methods that operate on the data structure

The first stage of writing an object-oriented program to solve a problem is to design the classes. This is part of object-oriented design. From this design, a program can be written using an object-oriented programming (OOP) language.

The programming languages the syllabus prescribes can be used for OOP: Python 3, VB.NET and Java.

Advantages of OOP over procedural languages

The advantage of OOP is that it produces robust, more reliable code. The attributes can only be manipulated using methods provided by the class definition. This means the attributes are protected from accidental changes. Classes provided in module libraries are thoroughly tested. If you use tried and tested building blocks to construct your program, you are less likely to introduce bugs than when you write code from scratch.

27.02 Designing classes and objects

When designing a class, we need to think about the attributes we want to store. We also need to think about the methods we need to access the data and assign values to the data of an object. A data type is a blueprint when declaring a variable of that data type. A class definition is a blueprint when declaring an object of that class. Creating a new object is known as 'instantiation'.

Any data that is held about an object must be accessible, otherwise there is no point in storing it. We therefore need methods to access each one of these attributes. These methods are usually referred to as **getters**. They get an attribute of the object.

When we first set up an object of a particular class, we use a constructor. A **constructor** instantiates the object and assigns initial values to the attributes.

Any attributes that might be updated after instantiation will need subroutines to update their values. These are referred to as **setters**. Some attributes get set only at instantiation. These don't need setters. This makes an object more robust, because you cannot change attributes that were not designed to be changed.

Python and VB.NET allow properties to be declared. A **property** combines the attribute with its associated setter and/or getter (see Figures 27.03 and 27.05).



KEY TERMS

Getter: a method to access its associated attribute

Constructor: a special type of method that is called to create a new object and initialise its attributes

Setter: a method to set the value of its associated attribute

Property: a class member that includes the attribute and also getter and setter method calls to access/set the attribute value

WORKED EXAMPLE 27.01

Creating a class

Consider the car data from Section 27.01.

When a car is manufactured it is given a unique vehicle ID that will remain the same throughout the car's existence. The engine size of the car is fixed at the time of manufacture. The registration ID will be given to the car when the car is sold.

In our program, when a car is manufactured, we want to create a new car object. We need to instantiate it using the constructor. Any attributes that are already known at the time of instantiation can be set with the constructor. In our example, `VehicleID` and `EngineSize` can be set by the constructor. The other attributes are assigned values at the time of purchase and registration. So, we need setters for them. The identifier table for the `Car` class is shown in Table 27.01.

Identifier	Data Type	Description
`Car`	`Class`	Class identifier
`VehicleID`	`STRING`	Unique ID assigned at time of manufacture
`Registration`	`STRING`	Unique ID assigned after time of purchase
`DateOfRegistration`	`DATE`	Date of registration
`EngineSize`	`INTEGER`	Engine size assigned at time of manufacture
`PurchasePrice`	`CURRENCY`	Purchase price assigned at time of purchase
`Constructor()`		Method to create a `Car` object and set properties assigned at manufacture
`SetPurchasePrice()`		Method to assign purchase price at time of purchase
`SetRegistration()`		Method to assign registration ID
`SetDateOfRegistration()`		Method to assign date of registration
`GetVehicleID()`		Method to access vehicle ID
`GetRegistration()`		Method to access registration ID
`GetDateOfRegistration()`		Method to access date of registration
`GetEngineSize()`		Method to access engine size
`GetPurchasePrice()`		Method to access purchase price

Table 27.01 Identifier table for `Car` class

We can represent this information as a class diagram in Figure 27.01.

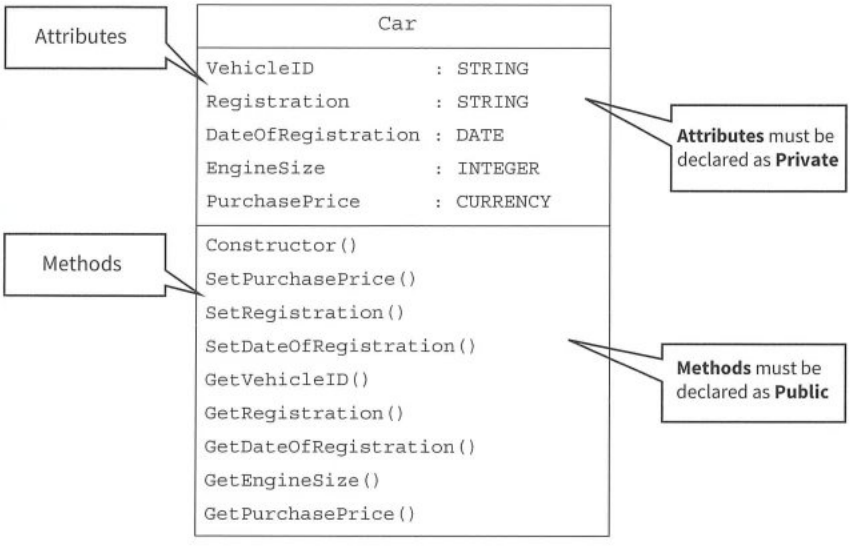


Figure 27.01 Car class diagram

27.03 Writing object-oriented code

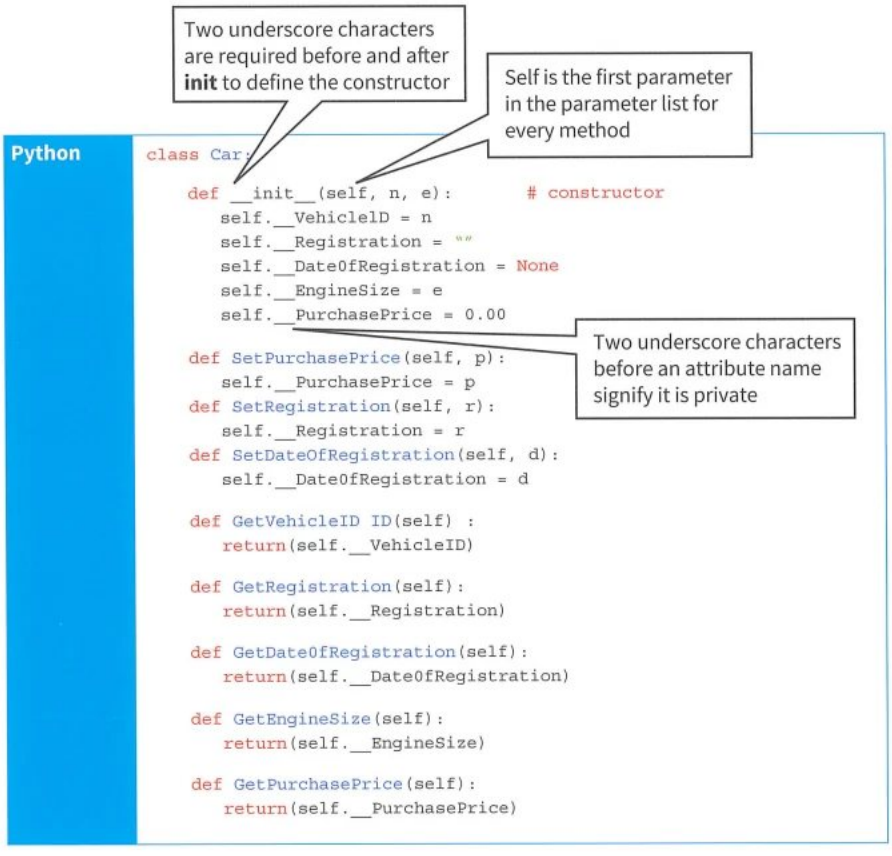
Declaring a class

Attributes should always be declared as 'Private'. This means they can only be accessed through the class methods. So that the methods can be called from the main program, they have to be declared as 'Public'. There are other modifiers (such as 'Protected'), but they are beyond the scope of this book.

The syntax for declaring classes is quite different for the different programming languages. We will look at the three chosen languages. You are expected to write programs in one of these.

Declaring a class in Python

The code below shows how a constructor, getters and setters can be declared in Python.



Python

```
class Car:
    def __init__(self, n, e):        # constructor
        self.__VehicleID = n
        self.__Registration = ""
        self.__DateOfRegistration = None
        self.__EngineSize = e
        self.__PurchasePrice = 0.00

    def SetPurchasePrice(self, p):
        self.__PurchasePrice = p
    def SetRegistration(self, r):
        self.__Registration = r
    def SetDateOfRegistration(self, d):
        self.__DateOfRegistration = d

    def GetVehicleID ID(self) :
        return(self.__VehicleID)

    def GetRegistration(self):
        return(self.__Registration)

    def GetDateOfRegistration(self):
        return(self.__DateOfRegistration)

    def GetEngineSize(self):
        return(self.__EngineSize)

    def GetPurchasePrice(self):
        return(self.__PurchasePrice)
```

Python also supports properties:

Python using properties

```
class Car:
    def __init__(self, n, e):        # constructor
        self.__VehicleID = n
        self.__Registration = ""
        self.__DateOfRegistration = None
        self.__EngineSize = e
        self.__PurchasePrice = 0.00

    @property
    def VehicleID(self):
        return(self.__VehicleID)

    @property
    def EngineSize(self):
        return(self.__EngineSize)

    @property
    def Registration(self):
        return(self.__Registration)

    @Registration.setter
    def Registration(self, r):
        self.__Registration = r

    @property
    def DateOfRegistration(self):
        return(self.__DateOfRegistration)

    @DataOfRegistration.setter
    def DateOfRegistration(self, d):
        self.__DateOfRegistration = d

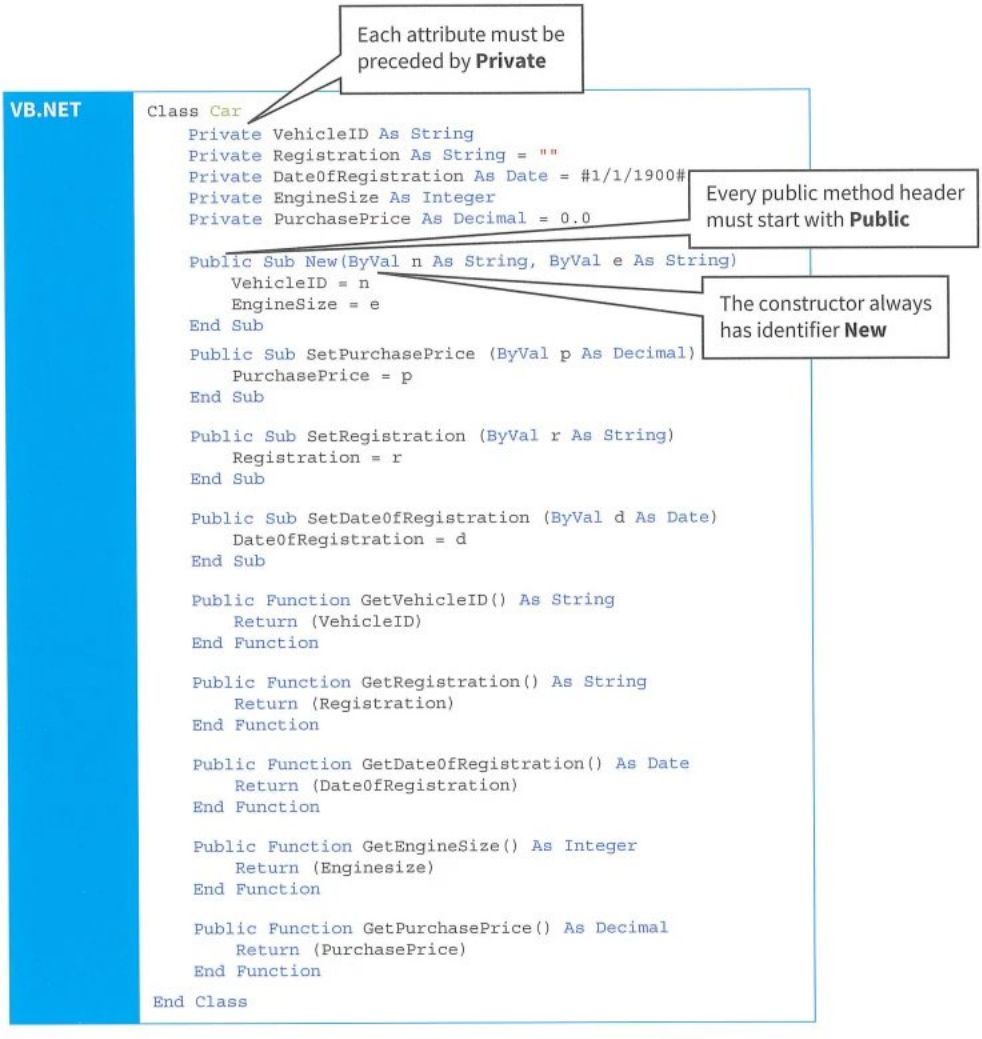
    @property
    def PurchasePrice(self):
        return(self.__PurchasePrice)

    @PurchasePrice.setter
    def PurchasePrice(self, p):
        self.__PurchasePrice = p
```

Note that not all properties provide a setter, only where attribute values are to be changed.

Declaring a class in VB.NET

The code below shows how attributes, the constructor, getters and setters can be declared in VB.NET.



VB.NET

```
Class Car
    Private VehicleID As String
    Private Registration As String = ""
    Private DateOfRegistration As Date = #1/1/1900#
    Private EngineSize As Integer
    Private PurchasePrice As Decimal = 0.0

    Public Sub New(ByVal n As String, ByVal e As String)
        VehicleID = n
        EngineSize = e
    End Sub
    Public Sub SetPurchasePrice (ByVal p As Decimal)
        PurchasePrice = p
    End Sub

    Public Sub SetRegistration (ByVal r As String)
        Registration = r
    End Sub

    Public Sub SetDateOfRegistration (ByVal d As Date)
        DateOfRegistration = d
    End Sub

    Public Function GetVehicleID() As String
        Return (VehicleID)
    End Function

    Public Function GetRegistration() As String
        Return (Registration)
    End Function

    Public Function GetDateOfRegistration() As Date
        Return (DateOfRegistration)
    End Function

    Public Function GetEngineSize() As Integer
        Return (Enginesize)
    End Function

    Public Function GetPurchasePrice() As Decimal
        Return (PurchasePrice)
    End Function
End Class
```

VB.NET also supports properties:

VB using properties

```
Module Module1
Class Car
    Public Property VehicleID() As String
    Public Property Registration() As String = ""
    Public Property DateOfRegistration() As Date = #1/1/1900#
    Public Property EngineSize() As Integer
    Public Property PurchasePrice() As Decimal = 0.0

    Public Sub New(ByVal n As String, ByVal e As String)
        VehicleID = n
        EngineSize = e
    End Sub

End Class
```

Note that getters and setters are automatically available for any property.
If some properties should be read-only, getters and setters have to be declared explicitly:

```
Class Car
  Private _VehicleID As String
  Public ReadOnly Property VehicleID() As String
    Get
      Return _VehicleID
    End Get
  End Property

  Public Property Registration() As String
  Public Property DateOfRegistration() As Date

  Private _EngineSize As Integer
  Public ReadOnly Property EngineSize() As Integer
    Get
      Return _EngineSize
    End Get
  End Property

  Public Property PurchasePrice() As Decimal
  Public Sub New(ByVal n As String, ByVal e As String)
    _VehicleID = n
    _Registration = ""
    _DateOfRegistration = #1/1/1900#
    _EngineSize = e
    _PurchasePrice = 0.0
  End Sub

End Class
```

Declaring a class in Java

The code below shows how attributes, the constructor and methods are declared in Java.

Java

```
import java.util.Date;
class Car
{
   private String vehicleID;
   private String registration;
   private Date dateOfRegistration;
   private int engineSize;
   private float purchasePrice;

   public Car(String n, int e)    // constructor
   {
      vehicleID = n;
      registration = "";
      dateOfRegistration = new Date();
      engineSize = e;
      purchasePrice = 0;
   }
   public void setPurchasePrice(float p)
   {
      purchasePrice = p;
   }
   public void setRegistration(String r)
   {
      registration = r;
   }
   public void setDateOfRegistration(Date d)
   {
      dateOfRegistration = d;
   }
   public String getVehicleID()
   {
      return(vehicleID);
   }
   public String getRegistration()
   {
      return(registration);
   }
   public Date getDateOfRegistration()
   {
      return(dateOfRegistration);
   }
   public int getEngineSize()
   {
      return(engineSize);
   }

   public float getPurchasePrice()
   {
      return(purchasePrice);
   }
}
```

Instantiating a class

To use an object of a class type in a program the object must first be instantiated. This means the memory space must be reserved to store the attributes.

The following code instantiates an object `ThisCar` of class `Car`.

Python	`ThisCar = Car("ABC1234", 2500)`
VB.NET	`Dim ThisCar As New Car("ABC1234", 2500)`
Java	`Car thisCar = new Car("ABC1234", 2500);`

Using a method

To call a method in program code, the object identifier is followed by the method identifier and the parameter list.

The following code sets the purchase price for an object `ThisCar` of class `Car`.

Python	`ThisCar.SetPurchasePrice(12000)` `ThisCar.PurchasePrice = 12000 # using properties`
VB.NET	`ThisCar.SetPurchasePrice(12000)` `ThisCar.PurchasePrice = 12000 ' using properties`
Java	`thisCar.setPurchasePrice(12000);`

The following code gets and prints the vehicle ID for an object `ThisCar` of class `Car`.

Python	`print(ThisCar.GetVehicleID())` `print(ThisCar.VehicleID) # using properties`
VB.NET	`Console.WriteLine(ThisCar.GetVehicleID())` `Console.WriteLine(ThisCar.VehicleID) ' using properties`
Java	`System.out.print(thisCar.getVehicleID());`

TASK 27.01

1 Copy the `Car` class definition into your program editor and write a simple program to test that each method works.

2 A business wants to store data about companies they supply. The data to be stored includes: company name, email address, date of last contact.

- **a** Design a class `Company` and draw a class diagram.
- **b** Write program code to declare the class. Company name and email address are to be set by the constructor and will never be changed.
- **c** Instantiate one object of this class and test your class code works.

27.04 Inheritance

The advantage of OOP is that we can design a class (a base class or a superclass) and then derive further classes (subclasses) from this base class. This means that we write the code for the base class only once and the subclasses make use of the attributes and methods of the base class, as well as having their own attributes and methods. This is known as **inheritance** and can be represented by an inheritance diagram (Figure 27.02).

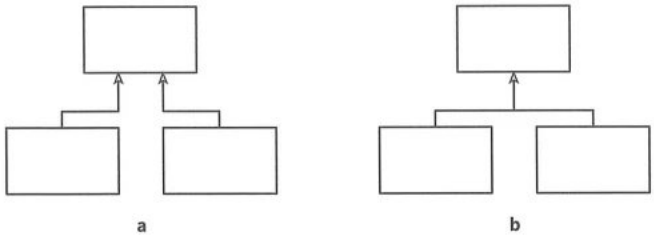


Figure 27.02 Inheritance diagram (a) standard and (b) alternative

KEY TERM

Inheritance: all attributes and methods of the base class are copied to the subclass

WORKED EXAMPLE 27.02

Implementing a library system

Consider the following problem.

- A college library has items for loan.
- The items are currently books and CDs.
- Items can be borrowed for three weeks.
- If a book is on loan, it can be requested by another borrower.

Table 27.02 shows the information to be stored.

Library item	
Book	**CD**
Title of book*	Title of CD*
Author of book*	Artist of CD*
Unique library reference number*	Unique library reference number*
Whether it is on loan*	Whether it is on loan*
The date the book is due for return*	The date the CD is due for return*
Whether the book is requested by another borrower	The type of music on the CD (genre)

Table 27.02 Library system information

The information to be stored about books and CDs needs further analysis. Note that we could have a variable `Title`, which stores the book title or the CD title, depending on which type of library item we are working with. There are further similarities (marked * in Table 27.02).

There are some items of data that are different for books and CDs. Books can be requested by a borrower. For CDs, the genre is to be stored.

We can define a class `LibraryItem` and derive a `Book` class and a `CD` class from it. We can draw the inheritance diagrams for the `LibraryItem`, `Book` and `CD` classes as in Figure 27.03.

(Continued)

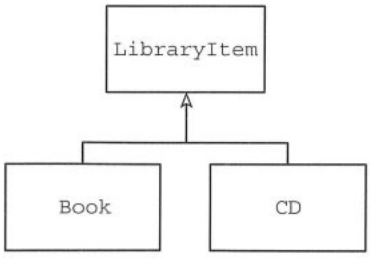


Figure 27.03 Inheritance diagram for LibraryItem, Book and CD classes

Analysing the attributes and methods required for all library items and those only required for books and only for CDs, we arrive at the class diagram in Figure 27.04.

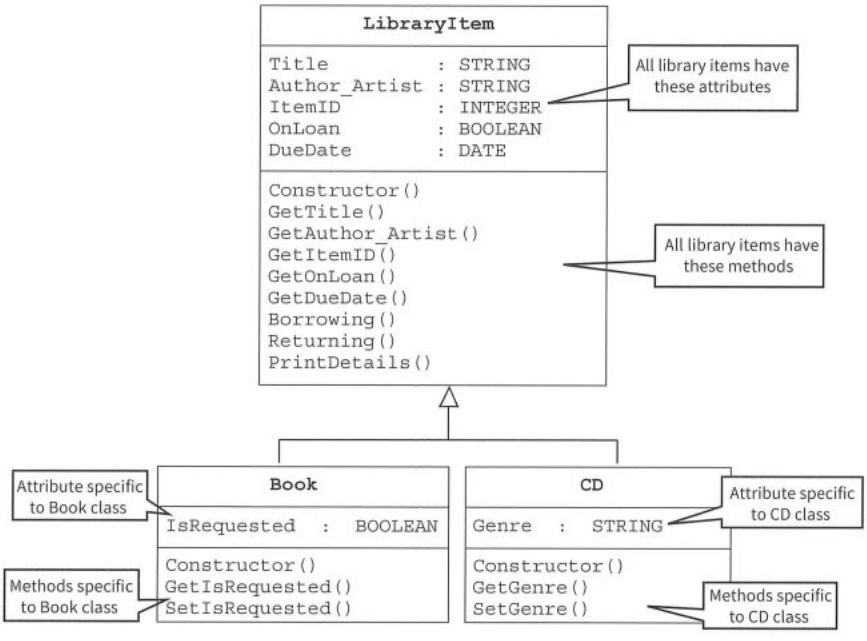


Figure 27.04 Class diagram for LibraryItem, Book and CD

A base class that is never used to create objects directly is known as an **abstract class**. `LibraryItem` is an abstract class.

KEY TERM

Abstract class: a base class that is never used to create objects directly

Declaring a base class and derived classes (subclasses) in Python

The code below shows how a base class and its subclasses are declared in Python.

Python

```
import datetime
class LibraryItem:
    def __ init__(self, t, a, i):            # initialiser method
        self.__Title = t
        self.__Author__Artist = a
        self.__ItemID = i
        self.__OnLoan = False
        self.__DueDate = datetime.date.today()

    def GetTitle(self):
        return(melf.__Title)

# other Get methods go here

    def Borrowing(self):
        self.__OnLoan = True
        self.__DueDate = self.__DueDate + datetime.timedelta(weeks-3)

    def Returning(self):
        self.__OnLoan - False

    def PrintDetails(self):
        print(self.__Title,',', self.__Author__Artist,',')
        print(self.__ItemID',', self.__OnLoan,',',Self,__DueDate)
class Book(Libraryltem):
    def __init__(self, t, a, i):             # initialiser method
        Libraryltem.__init__(self, t, a, i)
        self.__IsRequested = False
        self.__RequestedBy = 0

    def GetIsRequested(self):
        return(self.__IsRequested)

    def SetLiRequested(self):
        self.__IsRequested = True

class CD(LibraryItem):
    def __init__(self, t, a, i):             # initialiser method
        Libraryltem.__init__(self, t, a, i)
        self.__Genre = ''

    def GetGestre(self):
        return(self.__Genre)

    def SetGenre(self, g):
        self.__Genre = g
```

The base class definition

A subclass definition

This statement calls the constructor for the base class

Python using properties

```
class LibraryItem:
    def __init__(self, t, a, i):          # constructor / initialiser method
        self.__Title = t
        self.__Author__Artist = a
        self.__ItemID = i
        self.__OnLoan = False
        self.__DueDate = datetime.date.today()

    @property
    def Title(self):
        return(self.__Title)

# other getters go here

    def Borrowing(self):
        self.__OnLoan = True
        self.__DueDate = self.__DueDate + datetime.timedelta(weeks=3)

    def Returning(self):
        self.__OnLoan = False

    def PrintDetails(self):
        print(self.__Title,'; ', self.__Author__Artist,'; ', end='')
        print(self.__ItemID,'; ', self.__OnLoan,'; ', self.__DueDate)

class Book(LibraryItem):
    def __init__(self, t, a, i):          # initialiser method
        LibraryItem.__init__(self, t, a, i)
        self.__IsRequested = False
        self.__RequestedBy = 0

    @property
    def IsRequested(self):
        return(self.__IsRequested)

    @IsRequested.setter
    def IsRequested(self, b):
        self.__IsRequested = b

    # print details method for Book

    def PrintDetails(self):
        print("Book Details")
        LibraryItem.PrintDetails(self)
        print(self.__IsRequested)

class CD(LibraryItem):

    def __init __(self, t, a, i):         # initialiser method
        LibraryItem.__init__(self, t, a, i)
        self.__Genre = ""

    @property
    def Genre(self):
        return(self.__Genre)

    @Genre.setter
    def Genre(self, g):
        self.__Genre = g
```

Declaring a base class and derived classes (subclasses) in VB.NET

The code below shows how a base class and its subclasses are declared in VB.NET.

VB.NET

```
Class LibraryItem
    Private Title As String
    Private Author_Artist As String
    Private ItemID As Integer
    Private OnLoan As Boolean = False
    Private DueDate As Date = Today

    Sub Create(ByVal t As String, ByVal a As String, ByVal i As Integer)
        Title = t
        Author_Artist = a
        ItemID = i
    End Sub

    Public Function GetTitle() As String
        Return (Title)
    End Function

    ' other Get method, go here

    Public Sub Borrowing()
        OnLoan = True
        DueDate = DateAdd(DateInterval.Day, 21, Today()) '3 weeks from Today
    End Sub

    Public Sub Returning()
        OnLoan = False
    End Sub

    Sub PrintDetails()
        Console.WriteLine(Title & ", " & ItemID &  ", " & OnLoan &  ", " & DueDate)
    End Sub
End Class

Class Book
    Inherits LibraryItem
    Private IsRequested As Boolean = False
        Public Sub SetIsRequested()
            IsRequested = True
        End Sub

End Class

Class  CD
     Inherits  LibraryItem
     Private  Genre  As String  = ""

     Public  Function  GetGenre() As String
          Return  (Genre)
     End  Function

     Public  Sub  SetGenre(ByVal g As String)
         Genre = g
     End  Sub

End  Class
```

The base class definition

A subclass definition

The Inherits statement is the first statement of a subclass definition

VB.NET using properties

```
Class LibraryItem
    Public Property Title()
    Public Property Author_Artist As String
    Public Property ItemID As Integer
    Public Property OnLoan As Boolean = False
    Public Property DueDate As Date = Today

    Sub Create(ByVal t As String, ByVal a As String, ByVal i As Integer)
      Title = t
      Author_Artist = a
      ItemID = i
    End Sub

    Public Sub Borrowing()
      OnLoan = True
      DueDate = DateAdd(DateInterval.Day, 21, Today()) '3 weeks from today
    End Sub

    Public Sub Returning()
      _OnLoan = False
    End Sub

    Public Sub PrintDetails()
      Console.WriteLine(Title & "; " & ItemID & "; " & OnLoan & "; " & DueDate)
    End Sub
  End Class

  Class Book
    Inherits LibraryItem
    Public Property IsRequested() = False
  End Class

  Class CD
    Inherits LibraryItem
    Public Property Genre() As String
  End Class
```


Declaring a base class and derived classes (subclasses) in Java

The code below shows how a base class and its subclasses are declared in Java.

Java

```
import java.util.Date;
class LibraryItem
{
   private String title;
   private String author_Artist;
   private int itemID;
   private Boolean onLoan;
   private Date dueDate;
   public LibraryItem(String t, String a, int i)   // constructor
   {
```

Java

```
        title = t;
        author_Artist = a;
        itemID = i;
        onLoan = false;
        dueDate = new Date();
    }
    public String GetTitle()
    {
        return(title);
    }
// other Get methods go here
    public void borrowing()
    {
        onLoan = true;
        //dueDate = dueDate + 21;
    }
    public void returning()
    {
        onLoan = false;
    }

    public void printDetails()
    {
        System.out.print(title + " ; " + author_Artist + " ; ");
        System.out.println(itemID + " ; " + onLoan + " ; " + dueDate);
    }
}
class Book extends LibraryItem
{
    private Boolean isRequested;
    private int requestedBy;

    public Book(String t, String a, int i)  // constructor
    {
        super(t, a, i);
        isRequested = false;
        requestedBy = 0;
    }

    public Boolean getIsRequested()
    {
        return(isRequested);
    }
    public void setIsRequested()
    {
        isRequested = true;
    }
```

Java

```
}
class CD extends LibraryItem
{
   private String genre;

   public CD(String t, String a, int i)   // constructor
   {
      super(t, a, i);
      genre = "";
   }

   public String getGenre()
   {
      return(genre);
   }

   public void setGenre(String g)
   {
      genre = g;
   }
}
```

Instantiating a subclass

Creating an object of a subclass is done in the same way as with any class (See Section 27.03).

Python

```
ThisBook = Book(Title, Author, ItemID)
ThisCD = CD(Title, Artist, ItemID)
```

VB.NET

```
Dim ThisBook As New Book()
Dim ThisCD As New CD()
ThisBook.Create(Title, Author, ItemID)
ThisCD.Create(Title, Artist, ItemID)
```

Java

```
Book thisBook = new Book("Computing", "Sylvia", 1234);
CD thisCD = new CD("Let it be", "Beatles", 2345);
```

Using a method

Using an object created from a subclass is exactly the same as an object created from any class.

TASK 27.02

Copy the class definitions for `LibraryItem`, `Book` and `CD` into your program editor. Write the additional get methods. Write a simple program to test that each method works.

TASK 27.03

Write code to define a Borrower class as shown in the class diagram in Figure 27.05.

Borrower
BorrowerName : STRING EmailAddress : STRING BorrowerID : INTEGER ItemsOnLoan : INTEGER
Constructor() GetBorrowerName() GetEmailAddress() GetBorrowerID() GetItemsOnLoan() UpdateItemsOnLoan() PrintDetails()

Figure 27.05 Borrower class diagram

The constructor should initialise `ItemsOnLoan` to `0`.

`UpdateItemsOnLoan()` should increment `ItemsOnLoan` by an integer passed as parameter.

Write a simple program to test the methods.

27.05 Polymorphism

Look at Worked Example 27.02 and the code that implements it in Section 27.04. The constructor method of the base class is redefined in the subclasses. The constructor for the `Book` class calls the constructor of the `LibraryItem` class and also initialises the `IsRequested` attribute. The constructor for the `CD` class calls the constructor of the `LibraryItem` class and also initialises the `Genre` attribute.

The `PrintDetails` method is currently only defined in the base class. This means we can only get information on the attributes that are part of the base class. To include the additional attributes from the subclass, we need to declare the method again. Although the method in the subclass will have the same identifier as in the base class, the method will actually behave differently. This is known as **polymorphism**.



KEY TERM

Polymorphism: the method behaves differently for different classes in the hierarchy

The way the programming languages re-define a method varies.

The code shown here includes a call to the base class method with the same name. You can completely re-write the method if required.

Python

```
# define the print details method for Book
def PrintDetails(self):
    print("Book Details")
    LibraryItem.PrintDetails(self)
    print(self.__IsRequested)
```

This line calls the base class method with the same name.

VB.NET

```
' in base class, add the keyword Overridable
' to the method to be redefined
Overridable Sub PrintDetails()

' in subclass, add the redefined method:
Public Overrides Sub PrintDetails()
    Console.WriteLine("Book Details")
    MyBase.PrintDetails()
    Console.WriteLine(IsRequested)

End Sub
```

This line calls the base class method with the same name.

Java

```
@Override
public void printDetails()
{
    System.out.println("Book Details");
    super.printDetails();
    System.out.println(isRequested);
}
```

This line calls the base class method with the same name.

TASK 27.04

Use your program code from Task 27.02. Re-define the `PrintDetail` methods for the `Book` class and the `CD` class. Test your code.

TASK 27.05

Use your program code from Task 27.03. Add another attribute, `BorrowerID`, to the `LibraryItem` class so that the item being loaned can have the borrower recorded.

Change the `LibraryItem.Borrowing` and `LibraryItem.Returning` methods, so that `LoanItem.BorrowerID` and `Borrower.ItemsOnLoan` are updated when a library item is borrowed or returned.

TASK 27.06

Use your code from Task 27.02 or Task 27.04. Add another attribute, `RequestedBy`, to the `Book` class so that the borrower making the request can be recorded.

Change the method `Book.SetIsRequested`, so that `Book.RequestedBy` is updated when a book is requested.

TASK 27.07

Use your code from Task 27.06 to write the complete program to implement a simplified library system.

Write code to provide the user with a menu to choose an option. An example of a menu that would be suitable is shown in Figure 27.06.

```
1 - Add a new borrower
2 - Add a new book
3 - Add a new CD
4 - Borrow a book
5 - Return a book
6 - Borrow a CD
7 - Return a CD
8 - Request book
9 - Print all details
99 - Exit program
Enter your menu choice:
```

Figure 27.06 Library system menu

27.06 Garbage collection

When objects are created they occupy memory. When they are no longer needed, they should be made to release that memory, so it can be re-used. If objects do not let go of memory, we eventually end up with no free memory when we try and run a program. This is known as 'memory leakage'.

How do our programming languages handle this?

Python	Memory management involves a private heap containing all Python objects and data structures. The management of the Python heap is performed by the interpreter itself. The programmer does not need to do any housekeeping.
VB.NET	A garbage collector automatically reclaims memory from objects that are no longer referred to by the running program.
Java	The Java runtime environment deletes objects when it determines that they are no longer being used.

Table 27.03 Garbage collection strategies

27.07 Containment (aggregation)

In Section 27.04 we covered how a subclass inherits from a base class. This can be seen as generalisation and specialisation. The base class is the most general class, subclasses derived from this base class are more specialised.

We have other kinds of relationships between classes. **Containment** means that one class contains other classes. For example, a car is made up of different parts and each part will be an object based on a class. The wheels are objects of a different class to the engine object. The engine is also made up of different parts. Together, all these parts make up one big object.

KEY TERM

Containment: a relationship in which one class has a component that is of another class type

The containment relationship is shown in Figure 27.07.



Figure 27.07 Containment (aggregation) class diagram

WORKED EXAMPLE 27.03

Using containment

A college runs courses of up to 50 lessons. A course may end with an assessment. Object-oriented programming is to be used to set up courses. The classes required are shown in Figure 27.08.

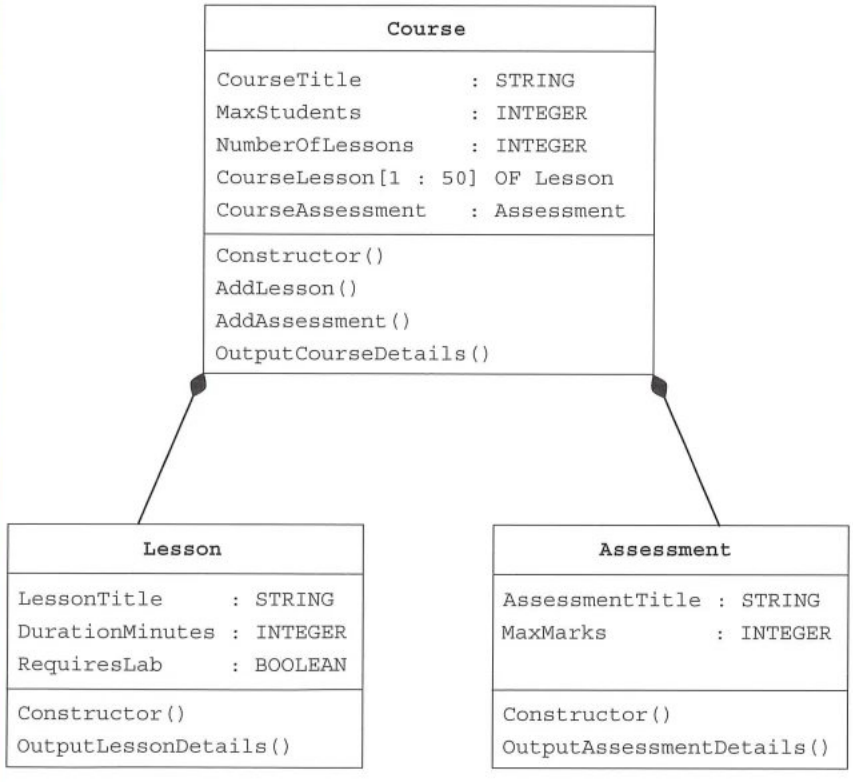


Figure 27.08 Containment class diagram

Assuming that all attributes for the `Lesson` and `Assessment` classes are set by values passed as parameters to the constructor, the code for declaring the `Lesson` and `Assessment` classes is straightforward.

The code below shows how the `Course` class is declared.

(Continued)

Python Course class declaration

```
class Course:
    def __init__(self, t, m): # sets up a new course
        self.__CourseTitle = t
        self.__MaxStudents = m
        self.__NumberOfLessons = 0
        self.__CourseLesson = []
        self.__CourseAssessment = Assessment

    def AddLesson(self, t, d, r):
        self.__NumberOfLessons = self.__NumberOfLessons + 1
        self.__CourseLesson.append(Lesson(t, d, r))

    def AddAssessment(self, t, m):
        self.__CourseAssessment = Assessment(t, m)

    def OutputCourseDetails(self):
        print(self.__CourseTitle, "Maximum number: ", self.__MaxStudents)
        for i in range(self.__NumberOfLessons):
            print(self.__CourseLesson[i].OutputLessonDetails())
```

VB.NET Course class declaration

```
Class Course
    Private CourseTitle As String
    Private MaxStudents As Integer
    Private NumberOfLessons As Integer = 0
    Private CourseLesson(50) As Lesson
    Private CourseAssessment As Assessment

    Public Sub Create(ByVal t As String, ByVal m As Integer)
        CourseTitle = t
        MaxStudents = m
    End Sub

    Sub AddLesson(ByVal t As String, ByVal d As Integer, ByVal r As Boolean)
        NumberOfLessons = NumberOfLessons + 1
        CourseLesson(NumberOfLessons) = New Lesson
        CourseLesson(NumberOfLessons).Create(t, d, r)
    End Sub
    Public Sub AddAssessment(ByVal t As String, ByVal m As Integer)
        CourseAssessment = New Assessment
        CourseAssessment.Create(t, m)
    End Sub

    Public Sub OutputCourseDetails()
        Console.WriteLine(CourseTitle & "Maximum number: " & MaxStudents)
        For i = 1 To NumberOfLessons
            CourseLesson(i).OutputLessonDetails()
        Next
    End Sub
End Class
```

(Continued)

Java Course class declaration

```
class Course
{
    private String courseTitle;
    private int maxStudents;
    private int numberOfLessons;
    private Lesson[] courseLesson;
    private Assessment courseAssessment;

    public Course(String t, int m)    // sets up a new course
    {
        courseTitle = t;
        maxStudents = m;
        numberOfLessons = 0;
        courseLesson = new Lesson[50];
        //courseAssessment = new Assessment();
    }
    public void addLesson(String t, int d, Boolean r)
    {
        numberOfLessons = numberOfLessons + 1;
        courseLesson[numberOfLessons] = new Lesson(t, d, r);
    }

    public void addAssessment(String t, int m)
    {
        courseAssessment = new Assessment(t, m);
    }
    public void outputCourseDetails()
    {
        System.out.println(courseTitle + " - Maximum number of students: " + maxStudents);
        for (int i = 1; i < numberOfLessons; i++)
        {
            courseLesson[i].outputLessonDetails();
        }
    }
}
```


Here are simple test programs to check it works.

Python test program

```
def Main():
    MyCourse = Course("Computing", 10) # sets up a new course

    MyCourse.AddAssessment("Programming", 100) # adds an assignment
    # add 3 lessons

    MyCourse.AddLesson("Problem Solving", 60, False)
    MyCourse.AddLesson("Programming", 120, True)
    MyCourse.AddLesson("Theory", 60, False)

    # check it all works
    MyCourse.OutputCourseDetails()
```

(Continued)

VB.NET test program

```
Dim MyCourse As New Course
MyCourse.Create("Computing", 10) ' sets up a new course
MyCourse.AddAssessment("Programming", 100) ' adds an assessment
' add 3 lessons
MyCourse.AddLesson("Problem Solving", 60, False)
MyCourse.AddLesson("Programming", 120, True)
MyCourse.AddLesson("Theory", 60, False)
'check it all works
MyCourse.OutputCourseDetails()
Console.ReadLine()
```

Java test program

```
Course myCourse = new Course("Computing", 10); // sets up a new course
myCourse.addAssessment("Programming", 100); // adds an assessment
// add 3 lessons
myCourse.addLesson("Problem Solving", 60, false);
myCourse.addLesson("Programming", 120, true);
myCourse.addLesson("Theory", 60, false);
// check it all works
myCourse.outputCourseDetails();
```


TASK 27.08

Write the code required for the `Lesson` and `Assessment` classes. Add the code for the `Course` class and test your program with the appropriate simple program from Worked Example 27.03.

Reflection Point:

Explain why a class can be regarded as a use-defined type.

Summary

- A class has attributes (declared as private) and methods (declared as public) that operate on the attributes. This is known as encapsulation.
- Python and VB.NET support properties: attributes that also include getters and setters.
- A class is a blueprint for creating objects.
- An object is an instance of a class.
- A constructor is a method that instantiates a new object.
- A class and its attributes and methods can be represented by a class diagram.
- Classes (subclasses) can inherit from another class (the base class or superclass). This relationship between a base class and its subclasses can be represented using an inheritance diagram.
- A subclass has all the attributes and methods of its base class. It also has additional attributes and/or methods.
- Polymorphism describes the different behaviour of a subclass method with the same name as the base class method.
- Containment is a relationship between two classes where one class has a component that is of the other class type. This can be represented using a containment diagram.

Exam-style Questions

1 A program is to be written using an object-oriented programming language. A bank account class is designed. Two subclasses have been identified:

- **Personal Account**: the account holder pays a monthly fee and may overdraw the account up to an agreed overdraft limit.
- **Savings Account**: the account holder must maintain a positive balance and gets paid interest on the balance at an agreed interest rate.

a Draw an inheritance diagram for these classes. [3]

The design for the `BankAccount` class consists of:

- attributes:
  - `AccountHolderName`
  - `IBAN`: International Bank Account Number
- methods:
  - `CreateNewAccount`
  - `SetAccountHolderName`
  - `GetAccountHolderName`
  - `GetIBAN`

b Write **program code** for the class definition of the superclass `BankAccount`. [5]

c **i** State the attributes and/or methods required for the subclass `PersonalAccount`. [4]

ii State the attributes and/or methods required for the subclass `SavingsAccount`. [4]

iii Identify the feature of object-oriented program design that combines the attributes and methods into a class. [1]

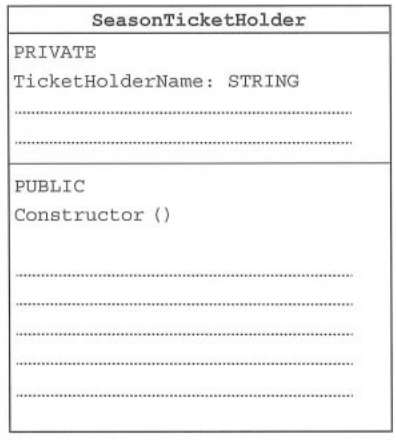
2 A bus company in a town has two types of season ticket for their regular customers: pay-as-you go and contract. All season ticket holders have their name and email address recorded.

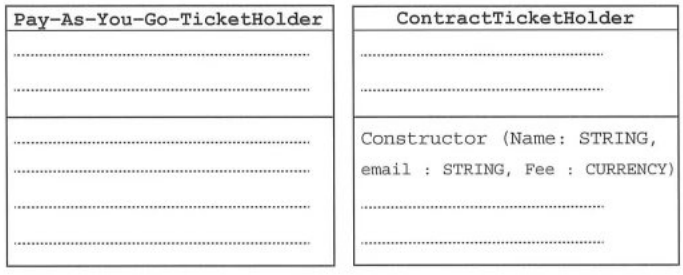
A pay-as-you-go ticket holder pays a chosen amount into their account. Each time the ticket holder makes a journey on the bus, the price of the fare is deducted from the amount held in the account. They can top up the amount at any time.

A contract ticket holder pays a fixed fee per month. They can then make unlimited journeys on the bus.

The bus company wants a program to process the season ticket data. The program will be written using an object-oriented programming language.

a Complete the class diagram showing the appropriate attributes and methods. [7]





b Attributes and methods can be declared as either public or private.

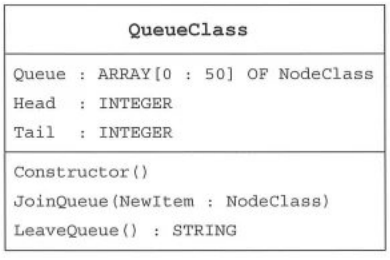
i Explain why the `SeasonTicketHolder` attributes are declared as private. [2]

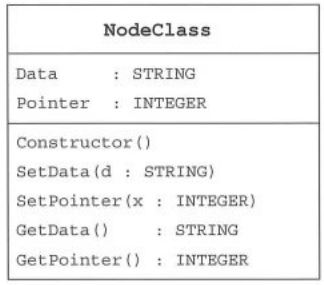
ii Explain why the `SeasonTicketHolder` methods have been declared as public. [2]

c Write program code to create a new instance of `ContractTicketHolder` with:

- Identifier: `NewCustomer`
- name: A. Smith
- email address: xyz@abc.xx
- monthly fee: $10 [3]

3 A queue abstract data type (ADT) is to be implemented using object-oriented programming. Two classes have been identified: `Queue` and `Node`. The class diagrams are as follows:





a State the relationship between these two classes. [1]

b The `NodeClass` constructor is to:

- create a new node
- initialise the `Data` attribute to the empty string
- initialise the `Pointer` attribute to -1.

Write program code to define `NodeClass`, including the get and set methods. [10]

c The `QueueClass` constructor is to:

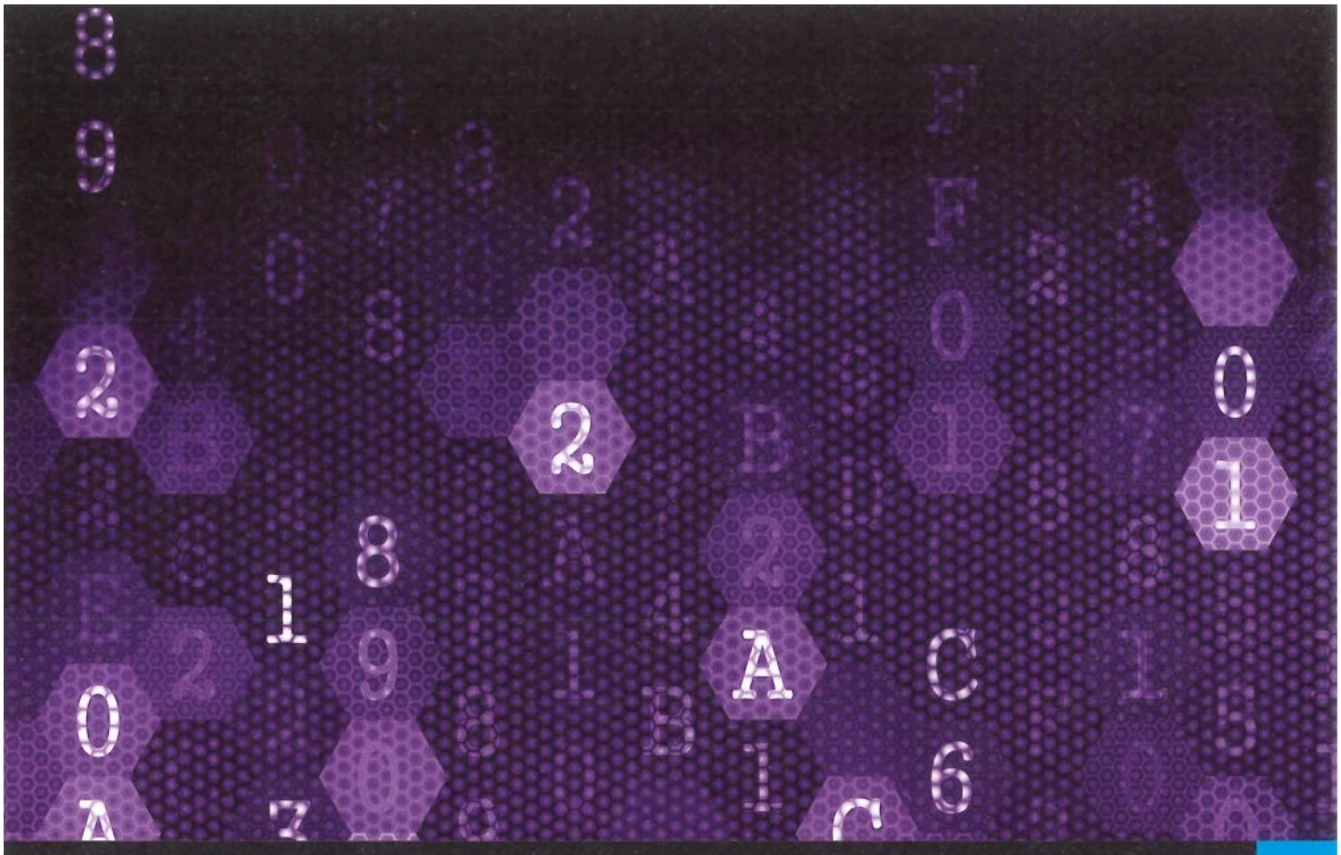
- create a new queue
- initialise the `Head` and `Tail` attributes to -1.

Write program code to define the constructor for `QueueClass`. [3]

d The `JoinQueue` method is to:

- create a new object, `Node`, of `NodeClass`
- assign the value passed as parameter to the `Data` attribute of `Node`
- assign `Node` to the end of `Queue`.

Write program code to define the `JoinQueue` method. [5]

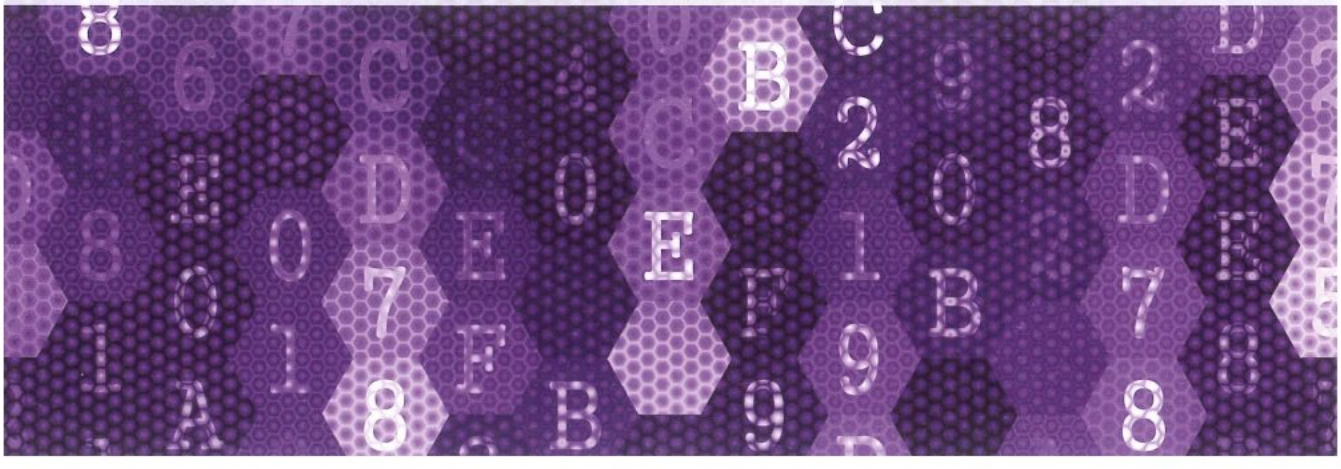


Chapter 28:
Low-level programming

Learning objectives

By the end of this chapter you should be able to:

- write low-level code that uses various address modes: immediate, direct, indirect, indexed and relative.



28.01 Processor instruction set

The instruction set we will use in this chapter is shown in Table 28.01.

Instruction			Explanation
Label	**Op code**	**Operand**	
Data movement instructions			
	LDM	#n	Immediate addressing. Load the denary number *n* to ACC
	LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
	LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
	LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
	LDR	#n	Immediate addressing. Load the denary number *n* to IX
	MOV	<register>	Move the contents of ACC to the given register (IX)
	STO	<address>	Store the contents of ACC at the given address
	STX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents from ACC to this calculated address*
	STI	<address>	Indirect addressing. The address to be used is at the given address. Store the contents of ACC at this second address*
*Note: The STX and STI instructions are not given as part of the instruction set in the syllabus for 9618			
Arithmetic operations			
	ADD	<address>	Add the contents of the given address to the ACC
	ADD	#n	Add the denary number *n* to the ACC
	SUB	<address>	Subtract the contents of the given address from the ACC
	SUB	#n	Subtract the denary number *n* from the ACC
	INC	<register>	Add 1 to the contents of the register (ACC or IX)
	DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
Comparison and jump instructions			
	JMP	<address>	Jump to the given address
	CMP	<address>	Compare the contents of ACC with the contents of <address>
	CMP	#n	Compare the contents of ACC with the denary number *n*
	CMI	<address>	Indirect addressing. The address to be used is at the given address. Compare the contents of ACC with the contents of this second address
	JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
	JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
Input/output instructions			
	IN		Key in a character and store its ASCII value in ACC
	OUT		Output to the screen the character whose ASCII value is stored in ACC
Bit manipulation instructions			
	AND	#n	Bitwise AND operation of the contents of ACC with the operand
	AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
	XOR	#n	Bitwise XOR operation of the contents of ACC with the operand
	XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
	OR	#n	Bitwise OR operation of the contents of ACC with the operand

Instruction			Explanation
Label	Op code	Operand	
	OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
	LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end
	LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end
Other			
	END		Return control to the operating system
<label>:	<opcode>	<operand>	Labels an instruction
<label>:	<data>		Gives a symbolic address <label> to the memory location with contents <data>

Table 28.01 Processor instruction set

In the assembly code in this chapter:

- ACC denotes the Accumulator
- IX denotes the Index Register
- # denotes immediate addressing
- B denotes a binary number, e.g. B01001010
- & denotes a hexadecimal number, e.g. &4A
- <address> can be an absolute address or a symbolic address.

An instruction set was introduced in Chapter 6.

28.02 Symbolic addresses

A label is a symbolic name for the memory location that it represents. You can treat it like a variable name. When writing low-level programs, we can give absolute addresses of memory locations. This is very restrictive, especially if we want to change the program by adding extra instructions. Writing low-level instructions using symbolic addresses (labels), allows us to think at a higher level. The assembler will allocate absolute addresses during the assembly process (see Chapter 6, Section 6.03).

28.03 Problem-solving and assembly-language programs

When writing a solution to a problem using low-level programming, we need to break down the problem into simple steps that can be programmed using the instruction set available.

One approach is to think in terms of the basic constructs we discussed for high-level languages. You can use the following examples as design patterns.

Assignment

Table 28.02 shows some examples of assembly language assignments that match the pseudocode.

Pseudocode examples	Assembly code examples	Explanation
`A ← 34`	`LDM #34` `STO A`	To store a value in a memory location, the value must first be generated in the accumulator

`B ← B + 1`	`LDD B`
	`INC ACC`
	`STO B`
		To increment the value stored at a memory location: first load the value into the accumulator, increment the value and then store the contents of the accumulator back to the memory location

`B ← B + A`	`LDD B`
	`ADD A`
	`STO B`
		To calculate a value: load the first value from a memory location into the accumulator, then add the value stored at the second memory location to the accumulator and then store the contents of the accumulator to the required memory location

`A ← -A`	`LDD A`
	`XOR #&FF`
	`INC ACC`
	`STO A`
	Alternative method:
	`LDM #0`
	`SUB A`
	`STO A`
		Load the value (assuming eight bits),
		`XOR` with `11111111` to produce the one's complement. Add 1 to get the two's complement.
		Alternatively, set the ACC to zero and then subtract the value from the ACC and store it back in the original address

Table 28.02 Using assignment instructions

TASK 28.01

Write assembly code instructions for this sequence of pseudocode statements:

```
A ← 2
B ← 10
C ← A + B
D ← A - B
```

Selection

Table 28.03 shows some examples of assembly language selections that match the pseudocode.

Pseudocode examples	Assembly code examples	Explanation

`IF A = 0`
`  THEN`
`  B ← B + 1`
`ENDIF`
	`  LDD A`
	`  CMP #0`
	`  JPN ENDIF`
	`THEN: LDD B`
	`  INC ACC`
	`  STO B`
	`ENDIF: ...`
		Load the contents of the memory location to be tested. Compare it with the required value (in this example 0). If the comparison result is false (A does not equal 0), a jump over the `THEN` part is required; if the comparison result is true (A = 0) then the following instructions are executed. For ease of understanding, the labels `THEN` and `ENDIF` are used.

`IF A = B`
`  THEN`
`  OUTPUT "Y"`
`  ELSE`
`  OUTPUT "N"`
`ENDIF`
	`  LDD A`
	`  SUB B`
	`  CMP #0`
	`  JPN ELSE`
	`THEN: LDM #89`
	`  OUT`
	`  JMP ENDIF`
	`ELSE: LDM #78`
	`  OUT`
	`ENDIF: ...`
		Load the contents of A.
		Subtract B.
		If the result is zero, A = B.
		If the comparison result is false (A does not equal B), jump to the ELSE part; if the comparison result is true (A = B) the instructions following the `THEN` label are executed.
		Note that a jump over the ELSE part is required.

Table 28.03 Using selection instructions

TASK 28.02

Write assembly code instructions for this sequence of pseudocode statements:

```
IF A <> 0
  THEN
    B ← A
  ELSE
    B ← B - 1
```

Repetition

Table 28.04 shows an example of repetition in assembly language that matches the pseudocode.

Pseudocode example	Assembly code example	Explanation
`A = 0`	`  LDM #0`	
`REPEAT`	`  STO A`	Store the initial value of the counter in A.
`  OUTPUT "*"`	`LOOP: LDM #42`	Generate the ASCII code for the character `"*"` and output it.
`  A ← A + 1`	`  OUT`	
`UNTIL A = 5`	`  LDD A`	Load the counter value,
	`  INC ACC`	increment the counter,
	`  STO A`	save it,
	`  CMP #5`	test for final value.
	`  JPN LOOP`	If final value has not been reached, jump back to beginning of loop.

Table 28.04 Using repetition instructions

TASK 28.03

Write assembly code instructions for this sequence of pseudocode statements:

```
Number ← 1
Total ← 0
Max ← 5
REPEAT
    Total ← Total + Number
    Number ← Number + 1
UNTIL Number = Max
```

Input/output

Table 28.05 shows some examples of input and output in assembly language that match the pseudocode.

Pseudocode examples	Assembly code examples	Explanation
`INPUT A`	`IN` `STO A`	Store a character input from the keyboard at memory location A.

`OUTPUT B`	`LDM #-1`
	`MOV IX`
	`LOOP: INC IX`
	`LDX B`
	`OUT`
	`CMP #13`
	`JPN LOOP`
		To output a string of characters stored in consecutive locations, starting at location labelled B, a loop and indexed addressing are used.
		The first time round the loop the index register is 0 and the character in memory location B will be loaded into the accumulator and output to the screen. Then a check is made for the end of the string (the carriage return character with ASCII code 13). If it is not the end of the string, jump back to the beginning of the loop.
		To output a number, the number must first be changed into its equivalent string and stored in consecutive memory locations. Then the above method can be used.

`INPUT A`	`LDM #-1`
	`MOV IX`
	`LOOP: INC IX`
	`IN`
	`STX A`
	`CMP #13`
	`JPN LOOP`
		Store a string of characters input from the keyboard into consecutive memory locations starting from A.
		Note: The STX instruction is not given as part of the instruction set in the syllabus for 9618 (from now on referred to as the 9618 instruction set). This means a string can not be saved with the 9618 instruction set.

Table 28.05 Using input and output instructions

TASK 28.04

Write assembly code instructions for this sequence of pseudocode statements:

```
Count ← 0
REPEAT
    Count ← Count + 1
    INPUT Character
UNTIL Character = "N"
```

TASK 28.05

Modify your assembly code instructions from Task 28.04 to implement this sequence of pseudocode statements:

```
Count ← 0
REPEAT
    Count ← Count + 1
    INPUT Character
UNTIL Character = "N"
OUTPUT Count
```

28.04 Absolute and relative addressing

An absolute address is the numeric address of a memory location. A program using absolute addresses cannot be loaded anywhere else in memory. Some assemblers produce relative addresses, so that the program can be loaded anywhere in memory.

Relative addresses are addresses relative to a base address, for example the first instruction of the program. When the program is loaded into memory the base address is stored in a base register BR. Instructions that refer to addresses then use the value in the base register, modified by the offset. For example, `STO [BR] + 10` will store the contents of the accumulator at the address calculated from (contents of the base register) + 10.

Table 28.06 shows an example of instructions using symbolic, relative and absolute addressing.

Symbolic addressing		Offset from base (START)	Relative addressing (base address stored in base register)	Absolute addressing	
START:	LDM #0	0	LDM #0	100	LDM #0
	STO A	1	STO [BR] + 10	201	STO 210
LOOP:	LDM #42	2	LDM #42	202	LDM #42
	OUT	3	OUT	203	OUT
	LDD A	4	LDD [BR] + 10	204	LDD 210
	INC ACC	5	INC ACC	205	INC ACC
	STO A	6	STO [BR] + 10	206	STO 210
	CMP #5	7	CMP #5	207	CMP #5
	JPN LOOP	8	JPN [BR] + 2	208	JPN 202
	END	9	END	209	END
A:	0	10	0	210	0

Table 28.06 Symbolic, relative and absolute addressing

It is very important that, at the end of the program, control is passed back to the operating system. Otherwise the binary pattern held in the next memory location will be interpreted as an instruction. If the content of that memory location does not correspond to a valid instruction, the processor will crash. The instruction END signals the end of the program instructions.

Note: the 9618 instruction set does not include relative (base register) addressing.

28.05 Indirect addressing

Indirect addressing is useful if the memory address to be used in an instruction is changed during the execution of the program.

One example is when programming subroutines to which parameters are passed by reference (this is beyond the scope of this book).

Another use of indirect addressing is for a pointer variable.

WORKED EXAMPLE 28.01

Writing a program for a simple queue

At the top level, we can write the problem using structured English:

```
Add a character to the queue:
1    Store the contents of the accumulator in the memory location
pointed to by the tail pointer.
2    Increment the tail pointer.
Remove a character from the queue:
1    Load contents of the memory location at the head of the queue.
2    Increment the head pointer.
```

Table 28.07 shows an example of instructions that implement the above queue-processing algorithms.

Instruction			Explanation
Label	**Op code**	**Operand**	
JOINQ:	STI	TAILPTR	Store contents of ACC in the memory location pointed to by the tail pointer
	LDD	TAILPTR	Increment the tail pointer
	INC	ACC	
	STO	TAILPTR	
	JMP	ENDQ	
LEAVEQ:	LDI	HEADPTR	Load contents of memory location at the head of the queue
	OUT		Output the character
	LDD	HEADPTR	Increment the head pointer
	INC	ACC	
	STO	HEADPTR	
	JMP	ENDQ	
ENDQ:			
HEADPTR:	QSTART		Pointer to start of queue
TAILPTR:	QSTART		Pointer to next free location in queue
QSTART:	""		Start of memory reserved for queue, currently empty

Table 28.07 Queue processing

Note that the value shown in Table 28.07 at the memory locations labelled HEADPTR and TAILPTR is the address of the start of the memory locations reserved for the queue. As values are added to the queue, the TAILPTR value will increase to point to the memory location at the end of the queue data. When a value is taken from the queue, the HEADPTR value will increase to point to the memory location at the head of the queue data.

Note: the 9618 instruction set does not include the STI opcode and a character can not be added to the queue using the 9618 instruction set.

TASK 28.06

Write instructions to reverse a word entered at the keyboard. This requires access to an area of memory treated as a stack.

Reflection Point:

Here are the addressing modes you should be able to use when writing low-level code:

Immediate	
Direct	
Indirect	
Indexed	
Relative	

Tick the box next to each addressing mode you can use easily and place a cross in the boxes next to the addressing modes you have difficulty with.

Summary

- A problem to be solved must be broken down into simple steps that can be programmed using the processor's given instruction set.
- A value must be copied into the accumulator before it can be processed.
- Processing includes:
  - arithmetic: adding, incrementing, decrementing
  - comparison: equal or not equal
  - bitwise operations: AND, OR, XOR, shifting
  - output to screen.
- To set a value in the accumulator it can be:
  - input from the keyboard
  - created using immediate addressing
  - loaded from a memory location using direct, relative, indirect or indexed addressing.
- An address can be absolute (a number) or symbolic (a label).

Exam-style Questions

1 The instruction set of a processor with one general-purpose register, the accumulator, includes the following instructions.

Instruction			Explanation
Label	**Op code**	**Operand**	
	`LDD`	`<address>`	Direct addressing. Load the contents of the given address to `ACC`
	`STO`	`<address>`	Store the contents of `ACC` at the given address
	`ADD`	`<address>`	Add the contents of the given address to the `ACC`
	`IN`		Key in a character and store its ASCII value in `ACC`
	`AND`	`<address>`	Bitwise `AND` operation of the contents of `ACC` with the contents of `<address>`

Instruction			Explanation
Label	Op code	Operand	
	LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end
	END		Return control to the operating system
<label>:	<data>		Gives a symbolic address <label> to the memory location with contents <data>

Key to the above table:

- ACC denotes the Accumulator.
- # denotes immediate addressing.
- & denotes a hexadecimal number, e.g. &4A.
- <address> can be an absolute address or a symbolic address.

a Explain the operation of the AND instruction. [1]

b The ASCII code for '0' is the binary value 00110000. The ASCII code for '1' is the binary value 00110001.

Write an AND instruction to convert any numeric digit stored in ACC in the form of an ASCII code to its eight-bit binary integer equivalent. [1]

c Write the assembly code instructions to convert a two-digit number keyed in at the keyboard to its BCD representation. Store the result in the memory location labelled Result. [7]

Instruction			Explanation
Label	Op code	Operand	
			Input first digit
			Convert from ASCII to its digit value
			Move to upper nibble
			Store in location Result
			Input second digit
			Convert from ASCII to its digit value
			Combine the two values
			Store result
			End of program
Mask:			Mask to convert from ASCII to digit equivalent
Result:	&00		Memory location for result

2 A given processor has one general-purpose register, the accumulator ACC, and one index register, IX. Part of the instruction set for this processor is as follows.

Instruction			Explanation
Label	Op code	Operand	
	LDM	#n	Immediate addressing. Load the number *n* to ACC
	LDD	<address>	Direct addressing. Load the contents of the given address to ACC
	LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC
	LDR	#n	Immediate addressing. Load the denary number n to IX

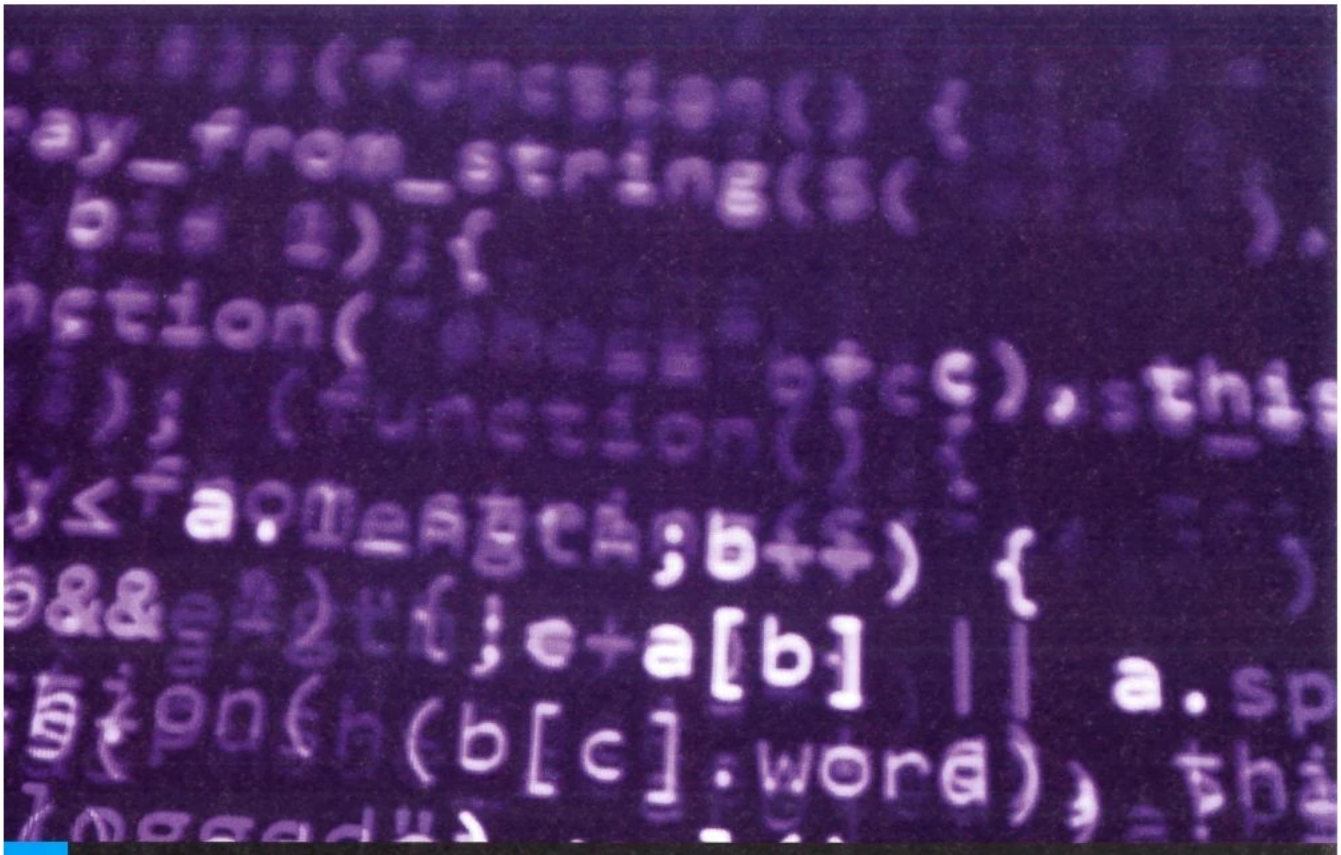
Instruction			Explanation
Label	Op code	Operand	
	STO	<address>	Store the contents of ACC at the given address
	STX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents from ACC to this calculated address
	ADD	<address>	Add the contents of the given address to the ACC
	INC	<register>	Add 1 to the contents of the register (ACC or IX)
	JMP	<address>	Jump to the given address
	CMP	<address>	Compare the contents of ACC with the contents of <address>
	CMP	#n	Compare the contents of ACC with the denary number *n*
	JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
	JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
	IN		Key in a character and store its ASCII value in ACC
	END		Return control to the operating system
<label>:	<op code>	<operand>	Labels an instruction
<label>:	<data>		Gives a symbolic address <label> to the memory location with contents <data>

Key to the above table:

- # denotes immediate addressing.
- <address> can be an absolute address or a symbolic address.

Write an assembly language program that outputs a sequence of characters stored in successive locations, starting at the location labelled: STRING. Output ends when the character in ACC is '!' (ASCII code 33). [8]

Instruction			Explanation
Label	Op code	Operand	
			Set index register to zero
			Load ACC with character stored at STRING (modified by index register)
			Increment index register
			Output character
			Is this character the ! key?
			No – jump to beginning of loop
			End of program
STRING:		72	String stored from here onwards
		69	
		76	
		80	
		33	

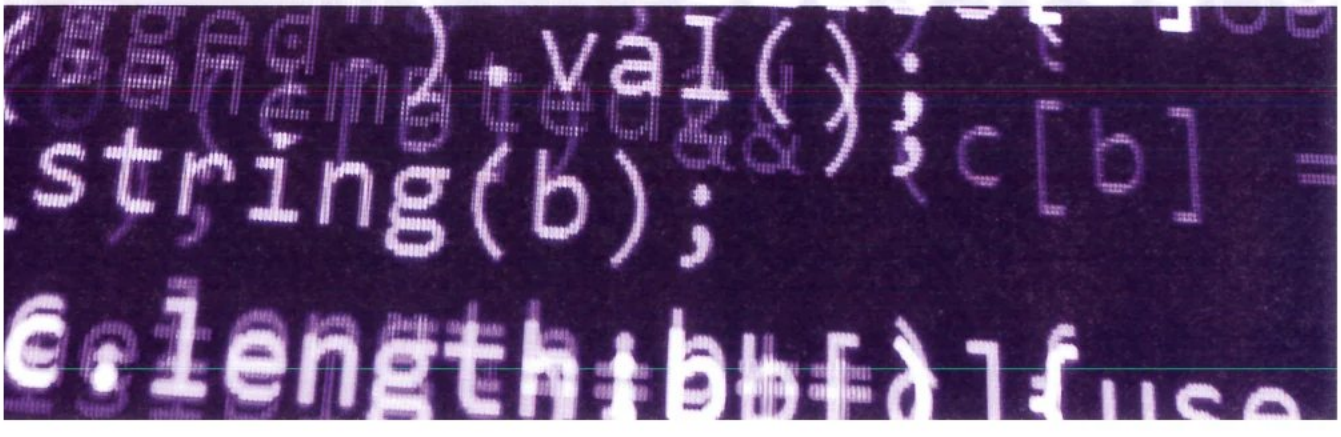


Chapter 29:
Declarative programming

Learning objectives

By the end of this chapter you should be able to:

- show understanding of and solve a problem by writing appropriate facts and rules based on supplied information
- show understanding of and write code that can satisfy a goal using facts and rules.



29.01 Declarative programming languages

Declarative languages include database query languages (such as SQL, see Chapter 11, Section 11.07), regular expressions, logic programming and functional programming.

Prolog is a logic programming language widely used for artificial intelligence and expert systems.

The Prolog programs in this chapter have been prepared using the SWI-Prolog environment shown in Figure 29.01 (see www.swi-prolog.org for a free download).

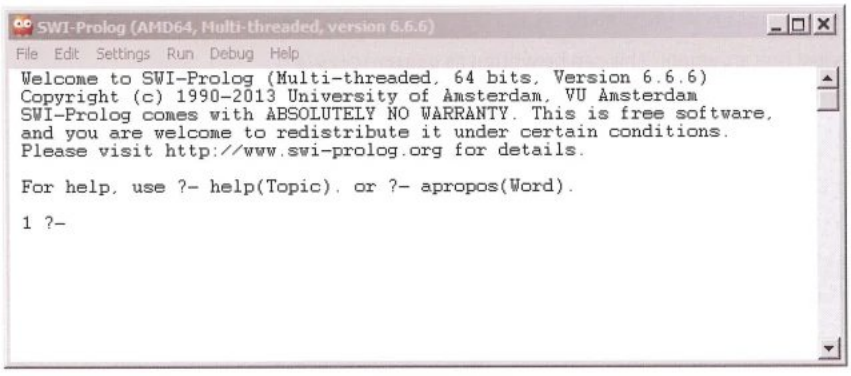


Figure 29.01 SWI-Prolog environment

29.02 Prolog basics

There are three basic constructs in Prolog: facts, rules and queries.

The program logic is expressed using clauses (facts and rules). Problems are solved by running a query (goal).

A collection of clauses is called a 'knowledge base'. Writing a Prolog program means writing a knowledge base as a collection of clauses. We use the program by writing queries.

A clause is of the form:

```
Head :- Body.
```

Note that a clause always terminates with a full stop (.)

Prolog has a single data type, called a 'term'. A term can be:

- an atom, a general-purpose name with no inherent meaning that always starts with a lower case letter
- a number, integer or float (real)
- a variable, denoted by an identifier that starts with a capital letter or an underscore (_)
- a compound term, a predicate, consisting of an atom and a number of arguments in parentheses ().

The arguments themselves can be compound terms. A predicate has an arity (that is, the number of arguments in parentheses).

Prolog is case sensitive.

29.03 Facts in Prolog

A clause without a body is a fact, for example:

```
01  capitalCity(paris).
02  capitalCity(berlin).
03  capitalCity(cairo).
```

The meaning of clause `01` is: Paris is a capital city.
`capitalCity(paris)` is a compound term. Both `capitalCity` and `paris` are atoms.
`capitalCity` is called a predicate and `paris` is the argument.

`capitalCity` has arity 1, as it has just one argument. This can be written as `capitalCity/1`, the `/1` showing that it takes one argument.

TASK 29.01

Launch the editor (File, New ...) from the SWI-Prolog environment. Enter the three clauses, as shown in Figure 29.02. Then save the file (File, Save buffer) as Ex1.

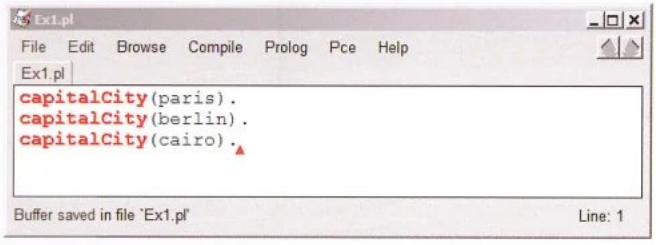


Figure 29.02 Example facts in SWI-Prolog editor

Clauses 01 to 03 are a knowledge base. We can run a query on this knowledge base.
To ask the question whether Paris is a capital city, we write:

```
capitalCity(paris).
```

Prolog answers `true`.
This means: yes, Paris is a capital city.
To ask the question whether London is a capital city, we write:

```
capitalCity(london).
```

Prolog answers `false`.
This means: no, London is not a capital city.
This is because the fact that London is a capital city has not been included in our knowledge base.

TASK 29.02

Run your own queries. You first need to consult the knowledge base (File, Consult ...) from within the Prolog environment. Note that SWI-Prolog uses the prompt ?- (see Figure 29.03).

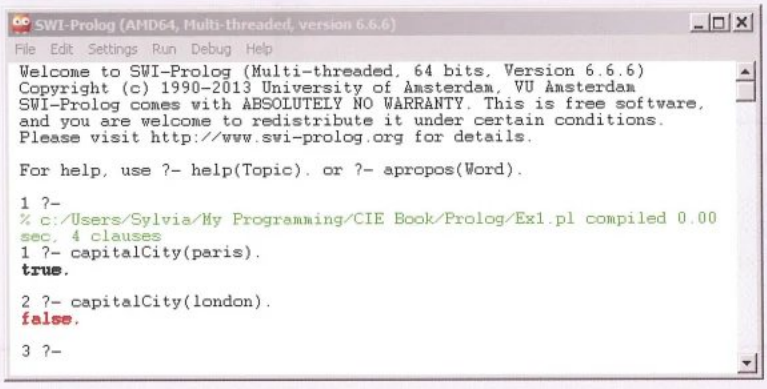


Figure 29.03 Example queries in SWI-Prolog

If your query does not get a response, check that you have:

- consulted your knowledge base (green text in Figure 29.03)
- used lower-case letters appropriately
- used a full stop at the end of your query.

29.04 Prolog variables

Let's add some more facts to our knowledge base. Comments in Prolog are enclosed in /* and */.

```
04  cityInCountry(paris, france). /* Paris is a city in France */
05  cityInCountry(berlin, germany).
06  cityInCountry(cairo, egypt).
07  cityInCountry(munich, germany).
```

To find out which country Berlin is in, we can run the query (see Figure 29.04):

```
cityInCountry(berlin, Country).
```

Note that `Country` is a variable (it starts with a capital letter).

To find out which cities are in Germany, we can run the query (see Figure 29.04):

```
cityInCountry(City, germany).
```

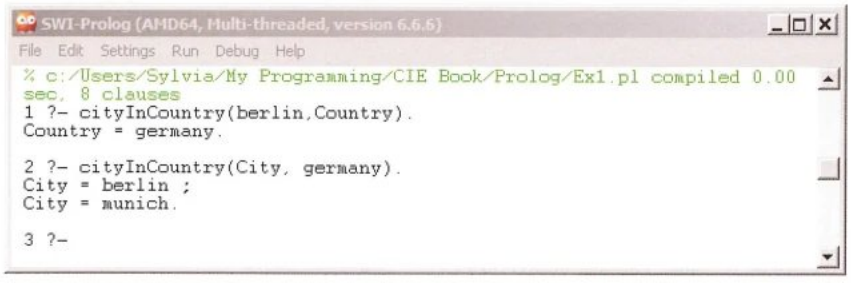


Figure 29.04 Instantiations of a variable

Note how Prolog responds when running a query that includes a variable. When there is more than one answer, you need to type a semicolon after the first answer and Prolog will give the second answer. The semicolon has the meaning OR. First `city` is instantiated to `berlin` and then `city` is instantiated to `munich`.

WORKED EXAMPLE 29.01

Using a knowledge base

Consider the following knowledge base:

```
01  vegetable(aubergine). /* aubergine is a vegetable */
02  vegetable(potato).
03  vegetable(tomato).
04  meat(chicken). /* chicken is a type of meat */
05  meat(beef).
06  meat(lamb).
07  ingredient(tagine, aubergine, 250). /* tagine contains 250g aubergine */
08  ingredient(tagine, tomato, 100).
09  ingredient(stew, beef, 400).
10  ingredient(stew, potato, 600).
```

We can check the ingredients of a tagine by asking:

```
ingredient(tagine, Ingredient, Amount).
```

Look at the response Prolog gives in Figure 29.05.

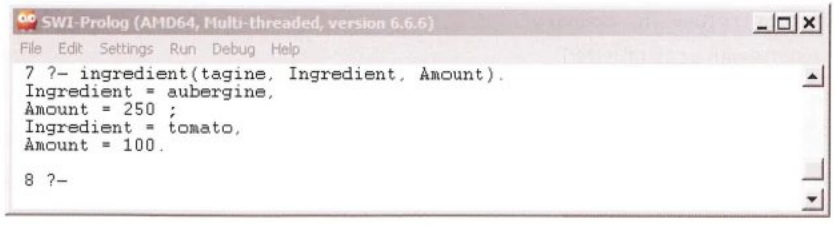


Figure 29.05 Instantiation of variables

29.05 The anonymous variable

Consider the knowledge base from Worked Example 29.01. If we are not interested in the amount of each ingredient, we can use the anonymous variable (represented by the underscore character). The query then becomes

```
ingredient(tagine, Ingredient, _).
```

29.06 Rules in Prolog

Remember a clause is of the form `Head :- Body.`

A rule's body consists of calls to predicates, which are called the rule's goals. A predicate is either true or false, based on its terms. If the body of the rule is true, then the head of the rule is true too.

WORKED EXAMPLE 29.02

Using rules in a knowledge base

Consider the following knowledge base:

```
01  parent(fred, jack). /* Fred is the father of Jack */
02  parent(fred, alia).
03  parent(fred, paul).
04  parent(dave, fred).
```

We know that G is a grandparent of S, if G is a parent of P and P is a parent of S.

We could write this as a rule:

```
grandparent(G, S) IF parent(G, P) AND parent(P, S).
```

However, in Prolog the `IF` is replaced by `:-` and the `AND` is replaced with a comma:

```
grandparent(G, S) :- parent(G, P), parent(P, S).
```

A person has a sibling (brother or sister) if they have the same parent. We can write this as the Prolog rule:

```
sibling(A, B) :-
   parent(P, A),
   parent(P, B).
```

If we run the query

```
sibling(jack, X).
```

we get the answers we expect, but we also get the answer that Jack is his own sibling. To avoid this, we modify the query to ensure that A is not equal to B:

```
sibling(A, B) :-
   parent(P, A),
   parent(P, B),
   not(A=B).
```

Question 29.01

What answer do you expect to get from Prolog to the following query?

```
sibling(dave, X).
```

TASK 29.03

Write a knowledge base for your own family. You can include more predicates, for example:

Predicate	Meaning
`male(fred).`	`Fred is male`
`female(alia).`	`Alia is female`

Write a rule for father.

Test your program.

WORKED EXAMPLE 29.03

Adding a rule to a knowledge base

Using the knowledge base from Worked Example 29.01, we want to know which dishes contain meat. We are not interested how much meat, so we don't need to know the value of the third argument of the predicate `ingredient/3`. We can write the rule:

```
containsMeat(X) :-
   ingredient(X, Meat, _),
   meat(Meat).
```

The query `containsMeat(X).` returns `X = stew.`

29.07 Instantiation and backtracking

Prolog responds to a query with an answer, such as the one in Worked Example 29.03:

```
X = stew.
```

The = sign is not an assignment as in imperative programs. The = sign shows instantiation.

How does Prolog use the knowledge base to arrive at the answers? One way to see exactly what Prolog is doing is to use the graphical debugger.

WORKED EXAMPLE 29.04

Use the knowledge base from Worked Example 29.03. After consulting the knowledge base, start the debugger (Debug, Graphical debugger) from the Prolog environment. Then type: `trace.` and then the goal as shown in Figure 29.06.

(Continued)

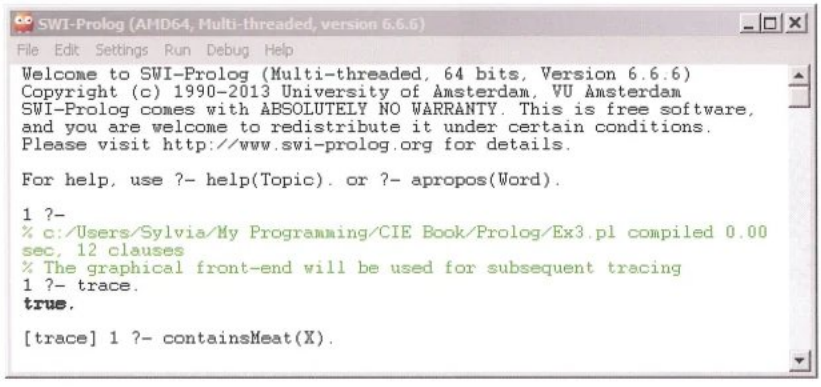


Figure 29.06 Switching on the trace facility

The graphical debugger window opens as shown in Figure 29.07.

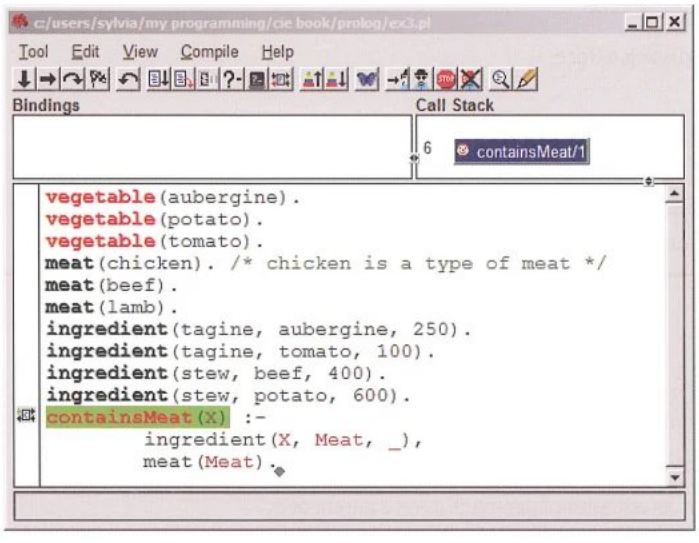


Figure 29.07 Graphical debugger

Using the space bar you can step through the program. When Prolog gives an answer in the Prolog Environment window, remember to input a semicolon, so that Prolog will go and check for another possible answer.

If you don't use the graphical debugger but type `trace.` you can see the trace in the SWI-Prolog window, as shown in Figure 29.08.

(Continued)

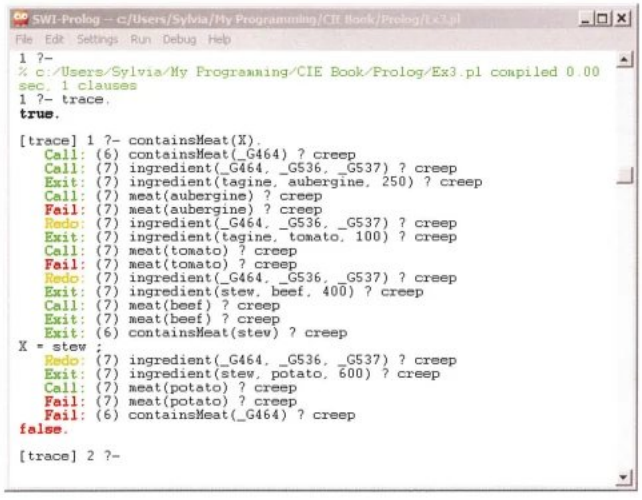


Figure 29.08 SWI-Prolog trace of goal `containsMeat(X)`

The following terminology is used when discussing a trace:

- `Call` is the initial entry to a predicate
- `creep` indicates that Prolog is moving to the next predicate
- `Exit` is a successful return
- `Redo` indicates that the predicate is backed into for another answer
- `Fail` indicates that Prolog can find no more solutions.

29.08 Recursion

Recursion for imperative languages is covered in Chapter 24. Recursion for declarative languages is where a rule is defined by itself, or more precisely, a rule uses itself as a sub-goal.

Let us expand the Family knowledge base from Worked Example 29.02.

We want a rule that defines whether person A is an ancestor of person B. If A is a parent of B, then A is an ancestor of B. Similarly, if person A is the parent of P, who is the parent of B, then A is an ancestor of B. This is true for the parent of a parent of a parent of B. In general, if A is a parent of X and X is an ancestor of B, then A is an ancestor of B. We can write this information as the rules shown in Figure 29.09.

```
ancestor(A, B) :- parent(A, B).
ancestor(A, B) :- parent(A, X), ancestor(X, B).
```

The base case

The general case

Figure 29.09 Recursive rules

Note that recursion in declarative programming must follow the equivalent rules that imperative programming must follow. A recursive rule must:

- have a base case
- have a general case
- reach the base case after a finite number of calls to itself.

TASK 29.04

Add the ancestor rules to the Family knowledge base and check that the following query gives the correct results:

```
ancestor(A, jack).
```

WORKED EXAMPLE 29.05

Creating the factorial function in Prolog

In Chapter 24, Worked Example 24.01, we programmed the factorial function using recursion with imperative programming. We can also program this function using recursion in Prolog.

```
factorial(0, 1).                  /* base case: 0! = 1    */
factorial(N, Result) :-           /* Result = N!          */
   M is N - 1,                    /* assign N-1 to M      */
   factorial(M, PartResult),      /* PartResult  = (N-1)! */
   Result is PartResult * N.      /* Result = N * (N-1)!  */
```


TASK 29.05

Enter the code from Worked Example 29.05 into the Prolog editor. Save it and consult it. Then pose the following query:

```
factorial(5, Answer).
```

Do you get the correct answer?

29.09 Lists

A list is an ordered collection of terms. It is denoted by square brackets with the terms separated by commas or in the case of the empty list, []. For example [1, 2, 3] or [red, green, blue]. An element can be any type of Prolog object. Different types can be mixed within one list. Lists are used in Prolog where arrays may be used in procedural languages.

Any non-empty list can be thought of as consisting of two parts: the head and the tail. The head is the first item in the list; the tail is the list that remains when we take the first element away. This means that the tail of a list is always a list.

Lists are manipulated by separating the head from the tail. The separator used is a vertical line (a bar):|

If Prolog tries to match `[H|T]` to `[car, lorry, boat, ship]`, it will instantiate `H` to `car` and `T` to `[lorry, boat, ship]`.

The clause definition `showHeadAndTail([H|T], H, T).` can be used to pose the query:

```
showHeadAndTail([fred, jack, emma], Head, Tail).
```

Prolog responds with:

```
Head = fred,
Tail = [jack, emma].
```

The clause definition `myList([1,2,3]).` can be used to pose the query:

```
myList([H|T]).
```

Prolog responds with:

```
H = 1,
T = [2, 3].
```

The clause definition `emptyList(A) :- A = [].` can be used to pose the query:

```
emptyList([1]).
```

Prolog responds with:

```
false.
```

List-processing predicate: `append`

The built-in predicate `append(A, B, C)` joins list A and list B and produces list C.

```
append([a, b], [c, d], MyList).
```

produces `MyList = [a, b, c, d].`

```
append(FirstList, [c, d], [a, b, c, d]).
```

produces `FirstList = [a, b].`

List-processing predicate: `member`

The built-in predicate `member(A, B)` returns true if item A is in list B.

```
member(c, [a, b, c, d, e]).
```

produces:

```
true.
```

And

```
member(X,[a, b, c, d]).
```

produces:

```
X = a ;
X = b ;
X = c ;
X = d.
```

List-processing predicates: `write` and `read`

The built-in predicate `write(A)` outputs A to the screen.

`write('message: ').` outputs `message: .`

`write(X).` outputs the value currently instantiated with the variable X.

The built-in predicate `read(A)` reads a value from the keyboard into variable A.

`read(Name).` waits for an atom to be input from the keyboard and instantiates the variable `Name` with that value.

Note that the input must start with a lower case letter and not have spaces or be enclosed in quotes.

`nl` moves the output to a new line.

We can write user-friendly programs using the `read` and `write` predicates.

WORKED EXAMPLE 29.06

Using the `read` and `write` predicates

Note how the interface with the user in the code below is written as a rule with the separate steps separated by commas (representing AND).

`assert/1` adds the clause given as the argument to the knowledge base.

`retractall/1` takes the given clause out of the knowledge base, so the next time the program is run, the new facts will be added and used in the goal.

```
/* Weather knowledge base */
weather(good):-
    temp(high),
    humidity(dry),
    sky(sunny).
weather(bad):-
    (humidity(wet);
    temp(low);
    sky(cloudy)).
/* interface */
go:-
    write('Is the temperature high or low? '),
    read(Temp), nl,
    write('Is the sky sunny or cloudy? '),
    read(Sky), nl,
    write('Is the humidity dry or wet? '),
    read(Humidity), nl,
    assert(temp(Temp)),
    assert(sky(Sky)),
    assert(humidity(Humidity)),
    weather(Weather),
    write('The weather is '), write(Weather),
    retractall(temp( _ ) ),
    retractall(sky( _ ) ),
    retractall(humidity( _ ) ).
```

To run the program, type `go`.

TASK 29.06

Test the recursively defined rule `writelist/1` to output the elements of a list.

```
writeList([]).
writeList([H|T]):-write(H), nl, writeList(T).
```

Reflection Point:

Can you explain the difference between a fact and a rule?

Summary

- Imperative programs reflect the steps of how to solve a problem.
- Declarative programs reflect what the problem is.
- A knowledge base consists of two types of clause: facts and rules.
- Clauses are sometimes referred to as predicates.
- The arity of a predicate shows how many arguments it takes.
- To solve a problem, the user of the knowledge base poses a query.
- A recursive rule is defined in terms of itself.
- In logic programming, a list is manipulated by separating the head from the tail ([H T]).

Exam-style Questions

1 A logic programming language is used to represent, as a set of facts and rules, details of cities of the world. The set of facts and rules are shown below in clauses labelled 1 to 17.

```
01  capital(vienna).
02  capital(london).
03  capital(santiago).
04  capital(caracas).
05  capital(tokyo).
06  cityIn(vienna, austria).
07  cityIn(santiago, chile).
08  cityIn(salzburg, austria).
09  cityIn(maracaibo, venezuela).
10  continent(austria, europe).
11  continent(chile, southAmerica).
12  continent(uk, europe).
13  continent(argentina, southAmerica).
14  iVisited(vienna).
15  iVisited(tokyo).
16  capitalOf(City, Country)
      IF capital(City) AND cityIn(City, Country).
17  europeanCity(City)
      IF cityIn(City, Country) AND continent(Country, europe).
```

These clauses have the following meanings:

Clause	Meaning
01	Vienna is a capital.
06	Vienna is in Austria.
10	Austria is in the continent of Europe.
14	I visited the city of Vienna.
16	`City` is the capital of `Country` if `City` is a capital and it is in `Country`.
17	`City` is a city in Europe if `City` is in `Country` and `Country` is in Europe.

a Write down the extra clauses needed to express the following facts:

i London is in the UK. [1]

ii I visited the city of Strasbourg. [1]

b The clause `cityIn(City, austria)` would return the result: `vienna, salzburg`.

Write down the result returned by the clause:

`continent(Country, southAmerica).` [2]

c Complete the rule to list countries that I have visited

`countriesIVisited(Country) IF ...` [3]

2 In a particular country, to become a qualified driver you must:

- have a licence: there is a minimum age at which a person can be issued with a licence and it is different for cars and trucks
- pass a theory test: it is the same test for cars and trucks
- pass a driving test for a specific vehicle (car or truck).

A declarative programming language is to be used to represent the knowledge base shown below:

```
01  minimumAge(car, 18).
02  minimumAge(truck, 21).
03  age(fred, 19).
04  age(jack, 22).
05  age(mike, 17).
06  age(jhon, 20).
07  age(emma, 22).
08  age(sheena, 19).
09  hasLicence(fred).
10  hasLicence(jack).
11  hasLicence(mike).
12  hasLicence(jhon).
13  hasLicence(emma).
14  hasLicence(sheena).
15  allowedToDrive(X, V)
        IF hasLicence(X) AND minimumAge(V, L)
            AND age(X, A)
            AND A >= L.
16  passedTheoryTest(jack).
17  passedTheoryTest(emma).
18  passedTheoryTest(jhon).
19  passedTheoryTest(fred).
20  passedDrivingTest(jhon, car).
21  passedDrivingTest(fred, car).
22  passedDrivingTest(jack, car).
23  passedDrivingTest(jack, truck).
24  passedDrivingTest(sheena, car).
25  qualifiedDriver(X, V)
        IF allowedToDrive(X, V)
            AND passedTheoryTest(X)
            AND passedDrivingTest(X, V).
```


These clauses have the following meanings:

Clause	Meaning
01	The minimum age for a car licence is 18.
03	Fred is aged 19.
09	Fred has a licence.
15	Person `X` is able to drive vehicle `V` if person `X` has a licence and the age `A` of person `X` is greater than or equal to the minimum age `L` to drive vehicle `V`.

a **i** Give **one** example of a fact in this knowledge base. [1]

ii Give **one** example of a rule in this knowledge base. [1]

b State the output produced from these clauses:

i `passedDrivingTest(Who, truck).` [1]

ii `allowedToDrive(mike, car).` [1]

iii `NOT(hasLicence(sheena)).` [1]

c Write a clause to output:

i all qualified car drivers [2]

ii all drivers who have passed the theory test but not a driving test. [3]

d To produce the output from a clause, the inference engine uses a process called backtracking.

Consider the clause:

```
AllowedToDrive(mike, car).
```

Identify the order in which clauses are used to produce the output. For each clause, describe the result that it returns. [5]